ChatGPT

# ASFireWire timing and recovery triage report

## Executive summary

The repository evidence strongly supports **three real bugs** behind the failures described in `AVC_RECOVERY_AND_SYNC_ALGO_AND_BUGS.md`, and it supports a fourth conclusion that one suspected cause is still **not proven**.

The first confirmed bug is a **callback-routing defect** introduced by FW-64 / merge commit `e9ea6ce`: `IsochService` exposes a single global `timingLossCallback_`, while `AudioCoordinator` constructs `dice_` before `avc_`. The DICE backend registers first, the AV/C backend registers second, and the AV/C registration overwrites the DICE one. After that, any receive timing-loss event is delivered only to AV/C, even when the active stream belongs to DICE. That is not a theoretical concern: the code shows a single slot in `IsochService`, both backends call `SetTimingLossCallback(...)`, and `AudioCoordinator` indeed constructs DICE before AV/C. [1]

The second confirmed bug is the **recovery-boundary violation** on the AV/C side. When AV/C decides a replay stall persisted long enough to be "real," it waits about **256 ms**, then calls `duplexCoordinator_.RecoverStreaming(...)`. That duplex path stops and restarts transport, but it does **not** run the normal AudioDriverKit producer reset/prefill sequence that `StartIO` uses. The normal start path explicitly resets the producer cursor with `ResetProducerForStart()`, prefills the transmit ring, and validates that `committedEnd == numSlots` before hardware streaming begins. Recovery bypasses that producer-owned reset, so stale `committedEnd` can survive into IT prime. The prime path then rejects out-of-range prefill counts. [2]

The third confirmed bug is an **audio data-exposure / packetizer underexposure defect**. `AmdtpPayloadWriter` classifies host frames that arrive after `Timeline().ExposedFrameEnd()` as `framesWithoutPacket` and simply skips them. Later, when RX replay becomes usable again, the TX side can move the producer cursor with `[TxAlign]` and appear to self-heal. This matches the document's description of "corrupt / click / silence while DMA and SYT remain alive, then self-rejoin without restart." In other words, the transport can look healthy while the audio content path is silently dropping host frames. [3]

What is **not yet proven** is that the original duplex sync problem is "raw RX/TX SYT mismatch." The document argues that ASFW already has the right local per-direction timing primitives—`ComputeReplaySytOffset()`, `ComputeReplaySyt()`, and the full-cycle timing helpers—and that raw cross-direction inequality is the wrong fault signal. The code and design notes are consistent with that assessment. So the repo evidence supports treating "RX/TX raw SYT inequality" as **unproven and likely a false lead** for the main audible failures. [4]

The most important practical conclusion is this: the repository does **not** point to one single defect. It points to a chain: **RX replay reset detection** → **wrong callback routing** for DICE, and separately **unsafe AV/C**

**restart escalation** → **stale TX producer state**; meanwhile, an independent **TX underexposure bug** can create audible loss and later "self-heal" even when transport telemetry looks fine. The document merged in PR #79 explicitly says that, at merge time, "streaming stability is broken for all devices/backends," which is consistent with the breadth of the architecture problem. [5]

## Repository structure and implementation map

The repository's public overview describes ASFireWire as a DriverKit-based FireWire stack for modern macOS, with OHCI, async transactions, AV/C, and an in-progress audio stack covering both AV/C and DICE devices. The timing/recovery work in question is concentrated under `ASFWDriver/Audio/...` and `ASFWDriver/Isoch/...`. [6]

From the primary repo artifacts, the relevant implementation map is:

| Area | Key files located | What they do |
|---|---|---|
| Audio orchestration | `ASFWDriver/Audio/Core/AudioCoordinator.cpp/.hpp` [7] | Owns backend selection and high-level audio device routing; currently constructs `dice_` before `avc_`. |
| AV/C backend and recovery | `ASFWDriver/Audio/Protocols/Backends/AVCAudioBackend.cpp/.hpp` [8] | Registers timing-loss callback, debounces RX replay loss, escalates to duplex restart. |
| DICE backend and recovery | `ASFWDriver/Audio/Protocols/Backends/DiceAudioBackend.cpp/.hpp` [9] | Registers its own timing-loss callback and routes into DICE restart handling. |
| Duplex stop/start/restart | `ASFWDriver/Audio/Protocols/Backends/AudioDuplexCoordinator.cpp` [10] | Coordinates stop/start/recovery, issues reconnect logic, but is below AudioDriverKit producer state. |
| Driver audio start path | `ASFWDriver/Audio/DriverKit/ASFWAudioDevice.cpp` [11] | Normal `StartIO` path resets TX producer state, binds slot providers, prefills ring, validates prefill, then starts streaming. |
| TX timeline / alignment | `ASFWDriver/Audio/DriverKit/ASFWAudioDriverZts.cpp` [12] | Logs and applies `[TxAlign]` frame-cursor alignment when replay / timing state changes. |

| Area | Key files located | What they do |
|---|---|---|
| Packetizer / payload writing | `ASFWDriver/Audio/Wire/AMDTP/AmdtpPayloadWriter.cpp` [13] | Writes host PCM into prepared packet slots; currently skips frames beyond exposure horizon. |
| RX replay / timing primitives | `ASFWDriver/Audio/Engine/Direct/Rx/DirectAudioReceiveConsumer.cpp`, `.../Wire/AMDTP/RxSequenceReplay.hpp`, `.../Wire/AMDTP/TimingUtils.hpp` [14] | Detects receive timing loss, resets replay epochs, stores replay history, computes replay SYT from RX sequence and timing helpers. |
| Transport / TX queue / transmit context | `ASFWDriver/Isoch/IsochService.cpp/.hpp`, `ASFWDriver/Isoch/Core/IsochTxQueue.hpp`, `ASFWDriver/Isoch/Transmit/IsochTransmitContext.cpp`, `ASFWDriver/Isoch/Transmit/IsochTxDmaRing.cpp` [15] | Provides the timing-loss callback slot, owns active duplex GUID, reads `committedEnd` for prefill, primes the descriptor ring, and rejects invalid prefill geometry. |

A particularly important structural fact is that `AudioCoordinator`'s member order in the header is `DiceAudioBackend dice_;` followed by `AVCAudioBackend avc_;`, and the constructor initializes them in that same order. That makes the callback overwrite deterministic, not incidental. [16]

## Relevant commits, pull requests, and issue signal

The repo history around July 18–19, 2026 is unusually concentrated and directly relevant. The visible commit log shows the research and stabilization work clustering around `e9ea6ce`, PRs `#76`, `#78`, and `#79`, plus supporting commits that moved receive interpretation, replay health, and telemetry boundaries. [17]

| Artifact | Date | Author | Relevance |
|---|---|---|---|
| Commit `e9ea6ce` / PR `#72` | visible on commit page; merged before Jul 18 history cluster [18] | mrmidi | Introduced FW-64 AV/C timing-loss debounce + restart, adding AV/C recovery subscription and `IsochService` timing-loss hooks—the change that created the single-slot overwrite hazard. [19] |
| Commit `8c4eb1b` | Jul 18, 2026 [20] | mrmidi | "wire receive consumers through service," relevant because it pushed receive lifecycle and callback routing through `IsochService`. [20] |
| Commit `7df7778` | Jul 18, 2026 [20] | mrmidi | "expose receive consumer lifecycle," relevant to replay-establishment and timing-loss signaling. [20] |

| Artifact | Date | Author | Relevance |
|---|---|---|---|
| Commit `a09a22d` | Jul 18, 2026 [20] | mrmidi | “introduce content-neutral receive seam,” relevant because timing-loss moved closer to transport/service boundaries. [20] |
| Commit `951abcc` | Jul 18, 2026 [20] | mrmidi | “move receive interpretation out of isoch,” relevant to callback and replay ownership boundaries. [20] |
| Commit `07dcae5` | Jul 18, 2026 [20] | mrmidi | “keep replay health on receive consumer,” directly relevant to first-fault classification. [20] |
| PR `#76` / commit `1c6112e`, merge `d16e2c4` | Jul 18, 2026 [21] | mrmidi | Refactored TX queue payload boundary, moving audio semantics out of transport and making later fix work safer. [22] |
| PR `#78` / commits `b83ae3c`, `5228347`, merge `e5e5fb2` | Jul 18, 2026 [23] | mrmidi | Added anomaly-only TX packetizer telemetry and the AV/C recovery / duplex-SYT design doc; explicitly records the “corrupt then self-rejoining” Duet behavior. [24] |
| Commit `cf6d8cc` | Jul 18, 2026 [25] | mrmidi + claude | “BUGLIST from the 2026-07-17 triage session,” indicating active bug triage just before the current research doc. [25] |
| Commit `fe86ccb` | Jul 18, 2026 [25] | mrmidi + claude | Detects session/device nominal-rate mismatch at RX qualification; supportive instrumentation rather than root fix. [25] |
| PR `#79` / commit `dd37f91`, merge `9055449` | Jul 19, 2026 [26] | mrmidi | Research branch capturing cursor geometry, replay horizon failure, FW-64 callback overwrite, and unsafe recovery; maintainer note says stability is broken for all devices/backends. [5] |

On the issue side, the public issue index visible from unsigned browsing shows open issues about iPod support, licensing, collaborator access, and other interfaces, but **no directly visible issue focused on timing-loss/recovery/packet loss**. The engineering discussion is therefore primarily in commits, PR descriptions, and the markdown design docs rather than in public issue threads. [27]

## Exact code paths behind the documented failures

### Callback overwrite and wrong-backend routing

The first half of the routing bug is the object construction order:

```
: publisher_(driver)
, dice_(...)
, avc_(...)
```

That is the actual `AudioCoordinator` constructor ordering in `AudioCoordinator.cpp` lines 994–1010. The matching header also declares `dice_` before `avc_`. [16]

The second half is that **both** backends subscribe to the same transport callback slot:

```
hostTransport_.SetTimingLossCallback([this](uint64_t guid) {
HandleRecoveryEvent(...); });
```

for DICE, and

```
hostTransport_.SetTimingLossCallback([this](uint64_t guid) {
HandleTimingLoss(guid); });
```

for AV/C. These registrations are in `DiceAudioBackend.cpp` lines 1838–1842 and `AVCAudioBackend.cpp` lines 1225–1231. [28]

`IsochService` confirms the slot is singular:

```
using TimingLossCallback = std::function<void(uint64_t guid)>;
TimingLossCallback timingLossCallback_{};
...
void SetTimingLossCallback(...) { timingLossCallback_ = ...; }
```

and delivery is a simple "if set, invoke for `activeGuid_`" path in `OnReceiveTimingLossDetected()`. That is the whole overwrite bug in code form: last writer wins. There is no router, no vector, no epoch, and no synchronization around replacement/invocation. The relevant lines are `IsochService.hpp` 748–856 and 1033–1038, plus `IsochService.cpp` 2394–2479. [29]

### Unsafe AV/C restart escalation

AV/C explicitly documents its strategy in code comments: debounce receive timing loss, then if replay is still not established, restart duplex. The debounce parameters are in `AVCAudioBackend.hpp`: `kTimingLossSettleMs = 256`, `kTimingLossPollMs = 32`, `kTimingLossMaxAttempts = 4`. [30]

The implementation does exactly that. After the settle loop, if `hostTransport_.IsReceiveReplayEstablished()` is still false, the code increments the attempt counter and calls:

```
const IOReturn status = duplexCoordinator_.RecoverStreaming(
    guid, DICE::DiceRestartReason::kRecoverAfterTimingLoss);
```

in `AVCAudioBackend.cpp` lines 1511–1644. [31]

`AudioDuplexCoordinator::RecoverStreaming()` acquires the GUID gate and hands off to `RunRecoveryStreaming()`, which, in turn, stops and restarts duplex via `RunDuplexStop(...)` and `RunDuplexStart(...)`. The critical lines are in `AudioDuplexCoordinator.cpp` 2835–2903 and 3170–3316. That confirms the document's central claim: the AV/C recovery path is transport/coordinator-centric, not AudioDriverKit-producer-centric. [32]

### Normal start path versus recovery path

The normal AudioDriverKit start path does the necessary producer reset work:

```
queueControl->ResetProducerForStart();
...
PrefillTxRingBeforeStart(ivars);
...
if (prefillExpose != expectedPrefill) failStart(...);
```

Those lines appear in `ASFWAudioDevice.cpp` around 2255, 2440, and 2451–2469. This is strong evidence that **producer reset and validated prefill are part of the intended start protocol**, and therefore must also exist in any correct recovery protocol. [33]

By contrast, IT start reads whatever `committedEnd` currently contains:

```
const uint64_t preFillCount = controlBlock_->committedEnd.load(...);
ring_.Prime(..., preFillCount);
```

from `IsochTransmitContext.cpp` lines 1851–1855. [34]

And prime explicitly refuses invalid counts:

```
if (preFillCount < numPackets || preFillCount > numSlots) { ... fail ... }
```

from `IsochTxDmaRing.cpp` lines 2075–2089. So if recovery reaches IT start with a stale producer cursor, the failure is entirely expected. [35]

### Packetizer underexposure and self-heal

The payload writer's behavior is explicit:

```
if (!timeline_->SnapshotSlotForAudioFrame(...)) {
    if (absoluteFrame >= timeline_->ExposedFrameEnd()) ++withoutPacket;
    else ++outsidePacket;
    continue;
}
```

That is in `AmdtpPayloadWriter.cpp` lines 791–835. The write path does not buffer the missed host frames for later packet exposure. It just classifies and skips them. [36]

Later, `ASFWAudioDriverZts.cpp` logs `[TxAlign] frame cursor -> ...` and the comments explicitly say the second and later alignment logs are the **self-heal closing a deficit that would otherwise be permanent silence**. That is very close to the report's observed symptom model and is one of the strongest pieces of corroborating evidence in the codebase. [37]

### RX replay reset semantics

The receive side logs first-fault records only when a replay epoch was already established. The reset reasons include `"packet-status"`, `"invalid-rx-timestamp"`, `"receive-cycle-gap"`, and `"syt-cadence-rejected"`. The reset log itself is emitted as `[RxReplayReset]` in `DirectAudioReceiveConsumer.cpp`, and the reason-name mapping appears in the same file. That supports the document's claim that a replay reset is **not equivalent to device outage**; it can be caused by several local RX validation failures. [38]

## Root-cause triage with severity and reproducibility

The best-supported root-cause classification is below.

| ID | Root cause | Evidence | Severity | Reproducibility | Triage |
|---|---|---|---|---|---|
| `AUDIO-RECOVERY-ROUTING-001` | Single `IsochService` timing-loss callback is overwritten by AV/C after DICE registration | `AudioCoordinator` construction order, both backends call `SetTimingLossCallback`, `IsochService` stores one `std::function`, and doc explicitly names FW-64/ `e9ea6ce` as the introducing change [39] | Critical | Deterministic whenever both backends exist in-process | **Confirmed root cause** |

| ID | Root cause | Evidence | Severity | Reproducibility | Triage |
|---|---|---|---|---|---|
| `AVC-RECOVERY-002` | AV/C auto-recovery restarts duplex transport without atomically quiescing/resetting/repriming the AudioDriverKit TX producer | AV/C debounce+restart code, normal `StartIO` reset/prefill path, IT prime depending on `committedEnd`, prime rejecting stale/out-of-range values [40] | Critical | High once RX replay reset persists past settle window | **Confirmed root cause** |
| `AVC-TX-EXPOSURE-001` | Host frames beyond `ExposedFrameEnd()` are dropped instead of retained until packet slots exist | `AmdtpPayloadWriter` increments `withoutPacket` and continues; `[TxAlign]` comment describes later self-heal of the deficit; PR #78 summary says Duet can DMA healthy packets while payload frames are beyond exposed timeline [41] | High for audio integrity | High under scheduler bursts / packetizer lag | **Confirmed root cause** |
| `AVC-HEALTH-001` | Original source of RX replay loss is still open: malformed packet, invalid RX timestamp, cycle gap, cadence rejection, or clock-anchor rejection | first-fault instrumentation and reset reasons show the failure class is real but not yet pinned to a single underlying source [42] | Medium to High | Unknown pending targeted fault injection / hardware reproduction | **Open root cause** |
| `AVC-SYNC-001` | Raw RX/TX SYT inequality as the primary bug | Document argues the opposite; repo already has direction-local replay/timing helpers and explicitly promotes expanded timeline comparison, not raw field equality [43] | Low confidence as causative fault | Not proven | **Not supported as primary cause** |

The most important triage judgment is that the **callback overwrite** and **unsafe recovery boundary** are not merely ancillary problems. They each create breakage even if the original RX disturbance were perfectly diagnosed. By contrast, the original RX disturbance remains open, but it is no longer the only—or even the most actionable—problem. [44]

## Prioritized fix plan

### P0 callback routing and event ownership

Move timing-loss ownership out of backend constructors and into `AudioCoordinator`. `IsochService` should expose **one coordinator-owned subscription**, but the payload should become a structured event, for example:

```
struct TimingLossEvent {
  uint64_t guid;
  ReplayResetReason reason;
  uint32_t rxEpoch;
  uint64_t sessionEpoch;
};
```

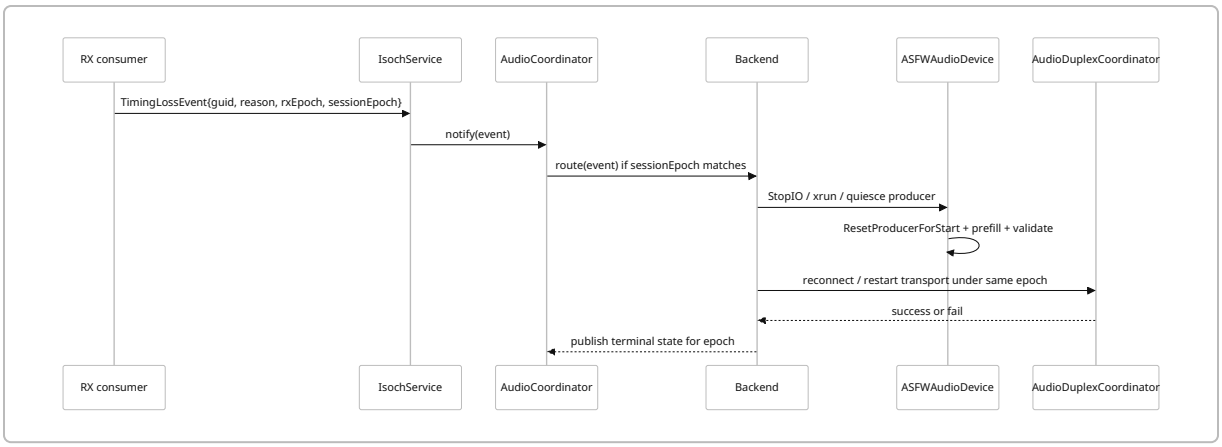
The event should be published by `DirectAudioReceiveConsumer` / `IsochService`, received by `AudioCoordinator`, and routed by `BackendForGuid(guid)` plus active session epoch. DICE and AV/C should no longer call `SetTimingLossCallback(...)` directly. Required files: `IsochService.hpp/.cpp`, `AudioCoordinator.hpp/.cpp`, `AVCAudioBackend.cpp/.hpp`, `DiceAudioBackend.cpp/.hpp`, and the RX consumer path that originates the event. This is low-to-medium effort, low risk, and should be done first because it removes deterministic misrouting. [45]

### P1 stop using background AV/C duplex restart as the automatic reaction

Until the producer and transport stop/start are unified under one epoch, AV/C should not call `duplexCoordinator_.RecoverStreaming(...)` from a background debounce worker. The safer interim behavior is: log the fault, mark the stream xrun / failed, and request an explicit restart from the audio side. This matches the document's "telemetry plus controlled xrun/explicit restart" recommendation and prevents the current half-running state. Required files: `AVCAudioBackend.cpp/.hpp`, possibly the audio runtime / nub restart path. Low effort, medium risk, because it intentionally removes automation and may increase explicit restart frequency. [46]

### P2 implement one recovery epoch across producer and transport

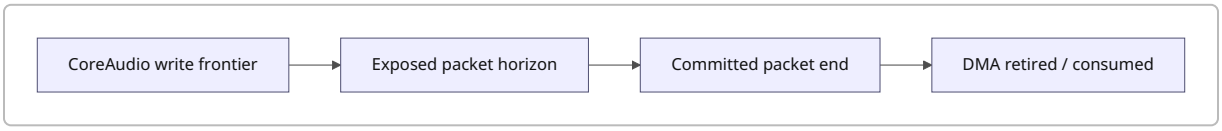
This is the core architectural repair. Recovery needs a single epoch handshake:




The rule should be: no IT prime, and no duplex restart, until the producer cursor and prefill are reset for the **same** epoch that owns the transport restart. That means `AudioDuplexCoordinator` cannot be the whole recovery story; it must be coordinated with `ASFWAudioDevice`'s producer-owned state. Required files: `ASFWAudioDevice.cpp`, `AudioDuplexCoordinator.cpp`, `AVCAudioBackend.cpp`, and any shared AudioDriverKit runtime state object. High effort, medium-to-high risk, but it is the real permanent fix for `AVC-RECOVERY-001` / `002`. [47]

### P3 fix the four-cursor TX data model

The TX side needs an explicit four-cursor model so "packet not yet exposed" does not mean "drop host audio."




Concretely, host frames that arrive ahead of `ExposedFrameEnd()` need a **pending** representation, not rejection. The packetizer's future packet horizon should be sized from the maximum I/O callback plus measured scheduler jitter, but it must not consume finite historical RX replay state as if it were future TX data; PR #79 explicitly says the 400-cycle preparation experiment did that incorrectly. Required files: `AmdtpPayloadWriter.cpp`, TX execution timeline / slot provider code, `ASFWAudioDriverZts.cpp`, and the simulator scripts referenced by PR #79. Medium-to-high effort, medium risk. [48]

### P4 finish first-fault instrumentation and add deterministic tests

Keep the anomaly-only logging, but add deterministic fault injection around RX timestamp/cadence failures and around producer/transport epoch mismatch. Also preserve the repo's current validation tools: `./build.sh --no-bump`, `ctest --test-dir build/tests_build --output-on-failure`, `python3 tools/tx_data_horizon_burst_sim.py suite`, and `python3 tools/buffer_geometry_verify.py verify --max-io 1024`. This is low-to-medium effort and low risk, but it is essential for making P0–P3 safe to merge. [49]

## Reproduction plan, validation tests, and implementation-risk table

The build environment, exact macOS version, entitlement state, and hardware inventory are **unspecified** in the available artifacts, so the reproduction plan has to be written as a repo-centric best-effort procedure rather than a guaranteed turnkey recipe. [50]

### Step-by-step reproduction plan

Start with a Debug build using the existing repository workflow:

1. Build with `./build.sh --no-bump`. That is the validation command cited in PRs `#76`, `#78`, and `#79`. [49]
2. Run host-side validation already referenced by the repo: `ctest --test-dir build/tests_build --output-on-failure`, `python3 tools/tx_data_horizon_burst_sim.py suite`, and `python3 tools/buffer_geometry_verify.py verify --max-io 1024`. [51]
3. For the callback overwrite bug, instantiate both backends against one `IsochService`, register DICE first and AV/C second, then inject `NotifyReceiveTimingLoss()` with a DICE active GUID. Current behavior should route only to AV/C because `timingLossCallback_` is last-writer-wins. [52]
4. For the stale-cursor recovery failure, reproduce a normal `StartIO`, then force `AVCAudioBackend::HandleTimingLoss()` down the path where replay remains stalled past the 256 ms settle interval. After `duplexCoordinator_.RecoverStreaming()`, inspect whether IT start reads an old `committedEnd`; failure is expected when prime reports “committed prefill … within slots” rejection. [53]
5. For the TX exposure bug, inject a scheduler burst or synthetic delay so host writes run ahead of `ExposedFrameEnd()`. The expected signature is nonzero `framesWithoutPacket`, followed later by a second `[TxAlign]` line indicating a self-heal. [54]
6. For RX first-fault diagnosis, force each reset reason class separately in `DirectAudioReceiveConsumer` and confirm one `[RxReplayReset]` record per established epoch with the correct reason string. [55]

### Proposed tests

| Test | Type | Pass condition |
|---|---|---|
| `IsochService_TimingLossRouter_DispatchesByGuidAndEpoch` | Unit | DICE and AV/C both receive only th own GUID/epoch events; no overwrite |
| `AVC_TimingLoss_NoAutoRestartWithoutProducerEpoch` | Unit/ integration | AV/C timing loss logs/xruns but do not invoke duplex restart unless producer epoch is prepared |
| `RecoveryEpoch_ResetProducerBeforePrime` | Integration | recovered start always sees `0 < committedEnd <= numSlots` an validated prefill |

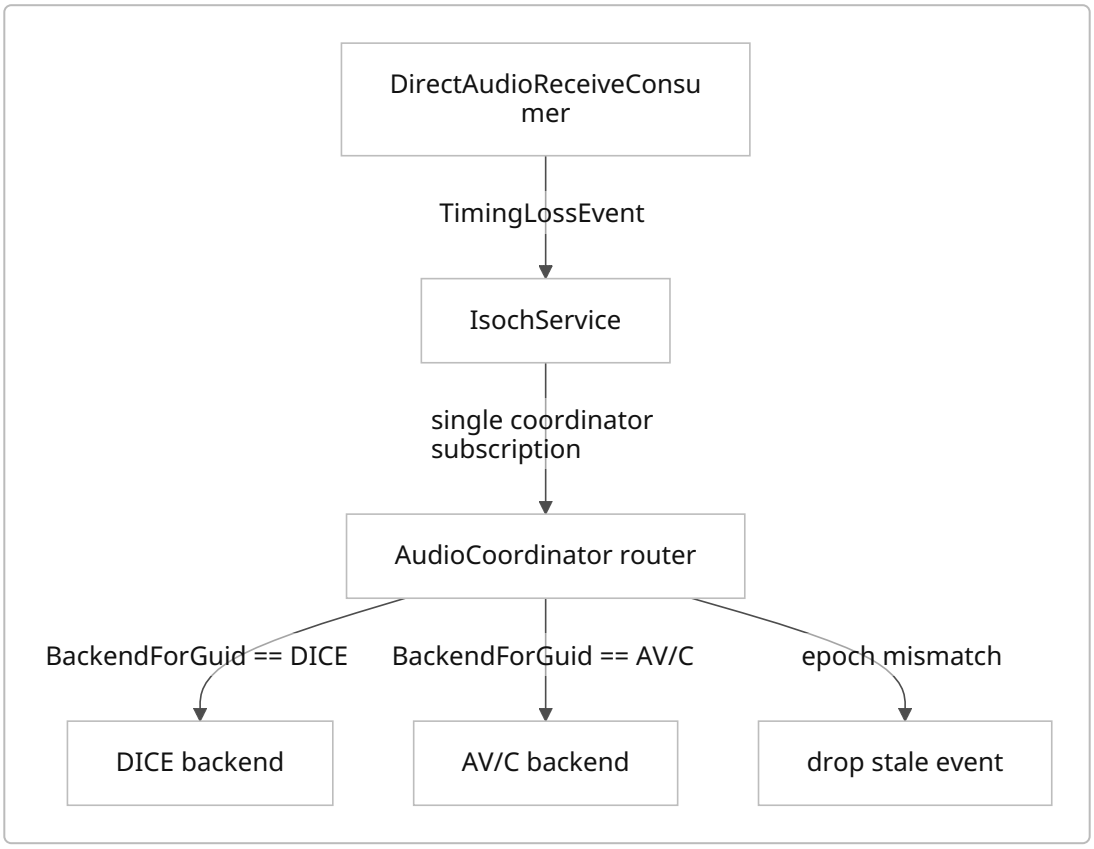
| Test | Type | Pass condition |
|---|---|---|
| `PayloadWriter_RetainsAheadFrames` | Unit | host frames beyond exposure become pending, not `framesWithoutPacket` loss |
| `TxAlign_NoSelfHealAfterDroppedFramesFix` | Integration/ log-based | with fix, no second anomaly-only `[TxAlign]` is needed to recover dropped content |
| `RxReplayReset_FirstFaultReasonMatrix` | Unit | packet-status / invalid-rx-timestam receive-cycle-gap / cadence-rejecte emit correct first-fault log and epo transition |
| `BurstSimulator_HorizonRegression` | Script/ regression | `tx_data_horizon_burst_sim.p` `suite` passes with new horizon model |
| `BufferGeometry_Verify_PostFix` | Script/ regression | `buffer_geometry_verify.py` `verify --max-io 1024` passes with no stale geometry assumptio |

### Candidate-fix comparison

| Priority | Fix | Main files | Effort | Risk | Why it ranks here |
|---|---|---|---|---|---|
| P0 | Replace single backend-owned callback with coordinator-owned routed event | `IsochService.*`, `AudioCoordinator.*`, `AVCAudioBackend.*`, `DiceAudioBackend.*` | Low–Med | Low | Removes a deterministic critical defect immediately. |
| P1 | Disable automatic AV/C background restart pending epoch-safe recovery | `AVCAudioBackend.*` | Low | Med | Stops current destructive behavior quickly. |
| P2 | Add atomic producer+transport recovery epoch | `ASFWAudioDevice.cpp`, `AudioDuplexCoordinator.cpp`, backend glue | High | Med–High | Fundamental architecture fix for stale-cursor restart path. |

| Priority | Fix | Main files | Effort | Risk | Why it ranks here |
|---|---|---|---|---|---|
| P3 | Rework TX horizon / pending-frame model | `AmdtpPayloadWriter.cpp`, timeline code, `ASFWAudioDriverZts.cpp` | Med–High | Med | Fixes the independent audible corruption path. |
| P4 | Expand fault injection and regression harnesses | RX consumer, host tests, `tools/*.py` | Low–Med | Low | Makes P0–P3 safer and keeps regressions from reappearing. |

### Callback routing target design




## Final assessment

The repository evidence supports the markdown document's core diagnosis with unusually high confidence. The **real problems** are not "mysterious sync drift" in the abstract; they are concrete ownership and synchronization bugs across the audio and transport boundary.

The first is a deterministic routing bug created by FW-64's callback design. The second is a recovery protocol bug that restarts transport without resetting the producer state that transport start assumes. The third is a packetizer/data-horizon bug that drops host frames when packet slots are not yet exposed, then later aligns the cursor and appears to recover. Those three together explain why the system can look transport-healthy but still produce corruption, silence, or backend-specific failure modes. [56]

What remains open is the original *trigger* for the receive replay reset. The repo now has enough first-fault instrumentation to separate "true device/no-RX outage" from "local receive validation failure," but not yet enough public evidence to say which one is dominant on the affected hardware. That means the right engineering posture is: fix P0–P3 first because they are already confirmed and user-visible, then use P4 instrumentation to finish the root-cause hunt for the underlying RX disturbance. [42]

In short: **the document is substantially correct**. The repo's current highest-value repairs are architectural —not "tune SYT math," but **route events correctly, unify recovery under one epoch, and stop dropping host audio when packet exposure lags**. [57]

---


[1] [2] [3] [4] [42] [43] [44] [46] [56] [57] ASFireWire/AVC_RECOVERY_AND_SYNC_ALGO_AND_BUGS.md at main · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/blob/main/AVC_RECOVERY_AND_SYNC_ALGO_AND_BUGS.md

[5] [48] Research: stream stability geometry and recovery telemetry by mrmidi · Pull Request #79 · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/pull/79

[6] mrmidi/ASFireWire: DriverKit OHCI FireWire Host ...
https://github.com/mrmidi/ASFireWire?utm_source=chatgpt.com

[7] [16] [39] [52] ASFireWire/ASFWDriver/Audio/Core/AudioCoordinator.cpp at main · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/blob/main/ASFWDriver/Audio/Core/AudioCoordinator.cpp

[8] [31] [40] [53] ASFireWire/ASFWDriver/Audio/Protocols/Backends/AVCAudioBackend.cpp at main · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/blob/main/ASFWDriver/Audio/Protocols/Backends/AVCAudioBackend.cpp

[9] [28] ASFireWire/ASFWDriver/Audio/Protocols/Backends/DiceAudioBackend.cpp at main · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/blob/main/ASFWDriver/Audio/Protocols/Backends/DiceAudioBackend.cpp

[10] [32] ASFireWire/ASFWDriver/Audio/Protocols/Backends/AudioDuplexCoordinator.cpp at main · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/blob/main/ASFWDriver/Audio/Protocols/Backends/AudioDuplexCoordinator.cpp

[11] [33] [47] ASFireWire/ASFWDriver/Audio/DriverKit/ASFWAudioDevice.cpp at main · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/blob/main/ASFWDriver/Audio/DriverKit/ASFWAudioDevice.cpp

[12] [37] ASFireWire/ASFWDriver/Audio/DriverKit/ASFWAudioDriverZts.cpp at main · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/blob/main/ASFWDriver/Audio/DriverKit/ASFWAudioDriverZts.cpp

13 36 41 54 ASFireWire/ASFWDriver/Audio/Wire/AMDTP/AmdtpPayloadWriter.cpp at main · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/blob/main/ASFWDriver/Audio/Wire/AMDTP/AmdtpPayloadWriter.cpp

14 38 55 ASFireWire/ASFWDriver/Audio/Engine/Direct/Rx/DirectAudioReceiveConsumer.cpp at main · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/blob/main/ASFWDriver/Audio/Engine/Direct/Rx/DirectAudioReceiveConsumer.cpp

15 ASFireWire/ASFWDriver/Isoch/IsochService.cpp at main · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/blob/main/ASFWDriver/Isoch/IsochService.cpp

17 20 21 23 25 26 Commits · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/commits/main

18 19 Merge pull request #72 from mrmidi/feature/fw-64-avc-timing-loss-reco… · mrmidi/ASFireWire@e9ea6ce · GitHub
https://github.com/mrmidi/ASFireWire/commit/e9ea6ce2fd60fc1e76efaf47fe9fa6feb3ddfaf4

22 49 51 refactor(isoch): make transmit queue payload opaque by mrmidi · Pull Request #76 · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/pull/76

24 50 Duet stability: packetizer telemetry and recovery notes by mrmidi · Pull Request #78 · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/pull/78

27 Issues · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/issues

29 45 ASFireWire/ASFWDriver/Isoch/IsochService.hpp at main · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/blob/main/ASFWDriver/Isoch/IsochService.hpp

30 ASFireWire/ASFWDriver/Audio/Protocols/Backends/AVCAudioBackend.hpp at main · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/blob/main/ASFWDriver/Audio/Protocols/Backends/AVCAudioBackend.hpp

34 ASFireWire/ASFWDriver/Isoch/Transmit/IsochTransmitContext.cpp at main · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/blob/main/ASFWDriver/Isoch/Transmit/IsochTransmitContext.cpp

35 ASFireWire/ASFWDriver/Isoch/Transmit/IsochTxDmaRing.cpp at main · mrmidi/ASFireWire · GitHub
https://github.com/mrmidi/ASFireWire/blob/main/ASFWDriver/Isoch/Transmit/IsochTxDmaRing.cpp